Spécifications Techniques : Modélisation Stochastique par Processus de Hawkes

Documentation Technique IA - Projet Météorage

Mars 2026

1 Introduction au Modèle Stochastique

Au lieu de traiter la fin d'orage comme un problème de classification binaire, nous la modélisons via la théorie des processus ponctuels. Un **Processus de Hawkes** est un processus de Poisson auto-excitant : l'occurrence d'un éclair augmente temporairement la probabilité d'en observer un nouveau. L'orage "meurt" lorsque cette probabilité retombe au niveau du bruit de fond.

2 Formalisation Mathématique des Données

2.1 L'Événement (Le Point)

Une alerte a_k n'est plus une matrice tabulaire, mais une **séquence d'événements marqués** dans un intervalle de temps $[0, T]$:

$$\mathcal{H}_t = \{(t_1, \mathbf{m}_1), (t_2, \mathbf{m}_2), \ldots, (t_n, \mathbf{m}_n)\} \quad \text{avec} \quad t_i < t \tag{1}$$

Où :

- t_i : l'instant précis du i-ème éclair depuis le début de l'alerte.
- $\mathbf{m}_i$: la "marque" de l'éclair (vecteur de caractéristiques). Ici, $\mathbf{m}_i = (d_i, c_i)^\top$ où d_i est la distance et $c_i \in \{0, 1\}$ indique s'il s'agit d'un intra-nuage (`icloud=True`) ou d'un nuage-sol[cite : 16, 17, 18].

2.2 Partition des Ensembles

La partition reste chronologique pour garantir la validité prédictive :

- $\mathcal{D}_{train} = \{\mathcal{H}^{(k)}\}_{k=1}^{M_{train}}$ (Alertes de 2016 à 2023).
- $\mathcal{D}_{test} = \{\mathcal{H}^{(k)}\}_{k=1}^{M_{test}}$ (Alertes de 2024 à 2025).

3 Le Modèle : La Fonction d'Intensité Conditionnelle

Le cœur du modèle est l'intensité conditionnelle $\lambda(t|\mathcal{H}_t)$, qui représente le taux d'occurrence attendu des éclairs à l'instant t, sachant le passé.

$$\lambda(t|\mathcal{H}_t) = \mu(t) + \sum_{t_i < t} \alpha(\mathbf{m}_i) e^{-\beta(t - t_i)} \tag{2}$$

- $\mu(t)$: Le taux d'occurrence de fond (probabilité qu'une nouvelle cellule se forme spontanément).
- $\alpha(\mathbf{m}_i)$: L'amplitude du "saut" d'excitation causé par l'éclair i. On paramètre α pour qu'un impact très proche ou nuage-sol génère plus d'excitation qu'un intra-nuage lointain.
- β : Le taux de décroissance exponentielle de l'orage (la vitesse de dissipation).

4 Inférence : Entrées et Sorties de la Fonction Finale

La fonction finale utilisée en production ne prédit pas un "label", mais une probabilité de sécurité sur un horizon temporel futur τ (par exemple, $\tau = 30$ minutes)[cite : 94].

4.1 Entrées (Inputs)

À l'instant t, la fonction prend en entrée l'historique récent de la cellule : $\mathcal{H}_t$.

4.2 Sorties (Outputs)

Le modèle produit $P_{fin}(t, \tau)$, la probabilité qu'**aucun éclair** ne survienne dans l'intervalle $[t, t + \tau]$:

$$P_{fin}(t, \tau) = \mathbb{P}(N(t, t + \tau) = 0|\mathcal{H}_t) \approx \exp\left(-\int_t^{t+\tau} \lambda(s|\mathcal{H}_s)ds\right) \tag{3}$$

L'algorithme calcule également le TTR (Temps Résiduel) en trouvant le plus petit Δt tel que $\lambda(t+\Delta t|\mathcal{H}_t) < \lambda_{seuil}$.

5 Entraînement : Estimation par Maximum de Vraisemblance (MLE)

Contrairement à la minimisation d'une erreur (comme XGBoost), le modèle de Hawkes "apprend" en maximisant la probabilité d'avoir observé les séquences réelles d'éclairs.

Pour une alerte donnée sur $[0, T]$, la log-vraisemblance $\mathcal{L}(\theta)$ pour les paramètres $\theta = (\mu, \alpha, \beta)$ est :

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} \log \lambda(t_i|\mathcal{H}_{t_i}) - \int_0^T \lambda(s|\mathcal{H}_s)ds \tag{4}$$

Nous utiliserons la librairie Python `tick` ou `scikit-survival` pour optimiser θ sur l'ensemble $\mathcal{D}_{train}$.

6 Règle de Décision (Le Prototype)

```python
def hawkes_decision_engine(history_t, params, tau_horizon=30):
    # 1. Calcul de l'intensite actuelle lambda(t)
    current_intensity = compute_lambda(history_t, params)

    # 2. Integration pour trouver la probabilite de 0 eclair dans les prochaines tau min
    p_safe = compute_survival_probability(current_intensity, params, tau_horizon)

    # 3. Decision
    if p_safe > 0.99:
        return "FIN D'ORAGE : Reprise securisee"
    else:
        # Calcul du temps requis pour que l'intensite chute
        ttr = compute_decay_time(current_intensity, params)
        return f"ATTENTE : TTR {ttr} min"
```